

Lab 6 Report - Design of a Simple General-Purpose Processor

Gina Bisoondial

COE328 Section 2

Dr. Geurkov

December 1, 2025

Table of Contents

Introduction.....	2
Components: Latch1, Latch2, 4:16 Decoder, FSM.....	3
ALU_1 for Problem Set 1 of the Lab 6 procedure.....	10
ALU_2 for Problem Set 2 of the Lab6 procedure.....	12
ALU_3 for Problem Set 3 of the Lab6 procedure.....	15
Conclusion.....	18
References.....	19

Introduction

The purpose of this lab was to design and construct an Arithmetic and Logic Unit (ALU) in VHDL on the Quartus 13.0 software to be implemented on an FPGA board. This was done by designings and building all the functions of the ALU, including all of its components, and to implement them using VHDL code. The processing unit is typically divided into four components, which were each thoroughly investigated in this project. There is the control unit, which performs the fetching of instructions and signals, the bus which controls the access to data values throughout the processing unit, registers which act as temporary storage unit, and the ALU core which performs arithmetic and logical operations based on the inputs and produces the desired outputs. The values used by the ALU were two 8-bit inputs A and B, which were derived from the last four digits of a student ID number.

$$A = (46)_{16} || B = (79)_{16}$$

$$A = (0100\ 0110)_2 || B = (0111\ 1001)_2$$

These are the 8-bit binary input values for A and B used throughout the duration of this lab.

Components: Latch1, Latch2, 4:16 Decoder, FSM

The components that are used in order to fulfill the requirements of this project are two latches, a 4:16 Decoder, and a FSM. A latch (or register) is a storage unit that temporarily stores the input bits, and then passes them onto other components in the system on the next rising edge of the clock signal (unless reset is set to modular 0, by which the latch does not operate). Two latches are used in this project, one to store the bits of A and one to store the bits of B. The FSM determines the pattern of the controller sequence through the nine different states (states 0 to 8 cycled consecutively, and then back to state 0 using the clock sequence input). The FSM has two

outputs, the current state and the student ID number. The nine student ID digits get displayed as sseg values on the FPGA, and the current state gets passed through the decoder sub-component. The 4-16 decoder is used to take the current states outputted by the FSM and convert them into the minterms values, to which will be inputted into the ALU core. These minterm values are called the microcodes, and they determine the operations that will be performed with the bits of A and B. Instead of directly integrating a 4-16 decoder, a design containing two 3-8 decoders, two AND gates, and one NOT gate was used, as shown in **Figure 3.2** below.

```

1  LIBRARY ieee ;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY latch1 IS
5  |   PORT ( A : IN STD_LOGIC_VECTOR(7 DOWNTO 0) ; --8 bit A input
6  |         Resetn, Clock : IN STD_LOGIC ; --1 bit clock input and 1 bit reset input bit
7  |         Q : OUT STD_LOGIC_VECTOR(7 DOWNTO 0) ) ;-- 8 bit output
8  | END latch1;
9  ARCHITECTURE Behavior OF latch1 IS
10 | BEGIN
11 |   PROCESS ( Resetn, Clock) --Process takes reset and clock as inputs
12 |   BEGIN
13 |     IF Resetn = '0' THEN -- when reset input is '0' the latches does not operate
14 |       Q <= "00000000" ;
15 |     ELSEIF Clock'EVENT AND Clock = '1' THEN -- level sensitive based on clock
16 |       Q <= A;
17 |     END IF;
18 |   END PROCESS;
19 | END Behavior;
20

```

Figure 1.1: VHDL Code for a Latch component.

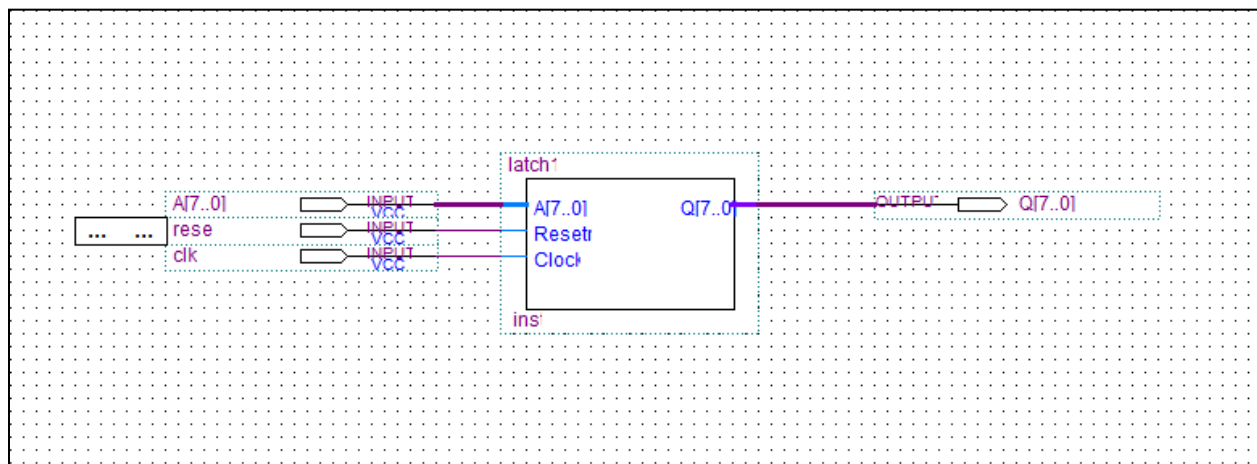


Figure 1.2: Block Diagram for Latch component.

Reset	Clock	Q
1	1	A
1	0	A
1	1	A
0	0	00000000

Figure 1.3: Truth Table for Latch component.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  entity machine is
4  |   port
5  |   (
6  |       clk      : in    std_logic;
7  |       data_in   : in    std_logic;
8  |       reset     : in    std_logic;
9  |       student_id : out   std_logic_vector(3 downto 0);
10 |       current_state: out  std_logic_vector(3 downto 0));
11 | end entity;
12 |
13 | architecture fsm of machine is
14 | |   type state_type is (s0, s1, s2, s3, s4, s5, s6, s7, s8, s_err);
15 | |   -- Register to hold the current state
16 | |   signal yfsm : state_type;
17 | begin
18 | |   process (clk, reset)
19 | |   begin
20 | |       if reset = '1' then
21 | |         yfsm <= s0;
22 | |       elsif (clk'EVENT AND clk = '1') then
23 | |         -- Determine the next state synchronously, based on
24 | |         -- the current state and the input
25 | |         case yfsm is
26 | |         when s0 =>
27 | |           if data_in='1' then yfsm <= s1;
28 | |           end if;
29 | |         when s1 =>
30 | |           if data_in='1' then yfsm <= s2;
31 | |           end if;
32 | |         when s2 =>
33 | |           if data_in='1' then yfsm <= s3;
34 | |           end if;
35 | |         when s3 =>
36 | |           if data_in='1' then yfsm <= s4;
37 | |           end if;
38 | |         when s4 =>
39 | |           if data_in='1' then yfsm <= s5;
40 | |           end if;

```

```

41         when s5 =>
42             if data_in='1' then yfsm <= s6;
43             end if;
44         when s6 =>
45             if data_in='1' then yfsm <= s7;
46             end if;
47         when s7 =>
48             if data_in='1' then yfsm <= s8;
49             end if;
50         when s8 =>
51             if data_in='1' then yfsm <= s0;
52             end if;
53         when others =>
54             yfsm <= s_err;
55         end case;
56     end if;
57 end process;
58
59 process (yfsm, data_in) -- data_in if reqd only
60 begin
61     case yfsm is
62         when s0 => student_id <= "0101";
63         when s1 => student_id <= "0000";
64         when s2 => student_id <= "0001";
65         when s3 => student_id <= "0011";
66         when s4 => student_id <= "0001";
67         when s5 => student_id <= "0100";
68         when s6 => student_id <= "0110";
69         when s7 => student_id <= "0111";
70         when s8 => student_id <= "1001";
71         when s_err => student_id <= "1110";
72     end case;
73 end process;
74     with yfsm select
75         current_state <= "0000" when s0,
76                         "0001" when s1,
77                         "0010" when s2,
78                         "0011" when s3,
79                         "0100" when s4,
80                         "0101" when s5,
81                         "0110" when s6,
82                         "0111" when s7,
83                         "1000" when s8,
84                         "1110" when s_err;
85 end architecture;

```

Figure 2.1: VHDL Code for Final State Machine.

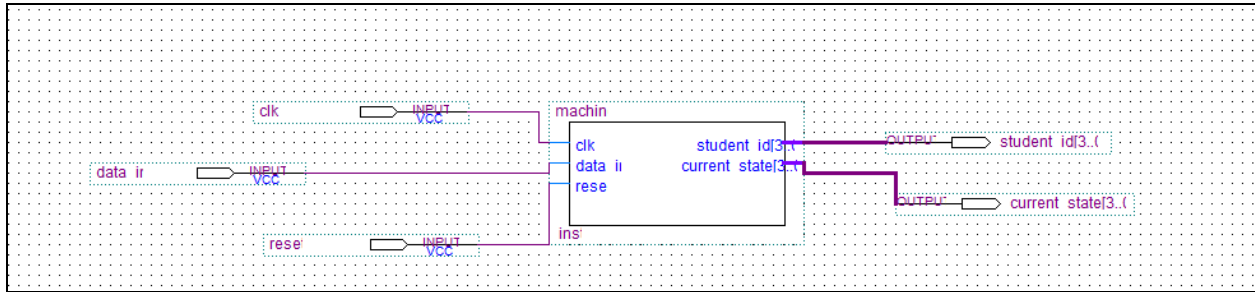


Figure 2.2: Block Diagram for Final State Machine.

Current State	Binary	data_in	Next State	Binary
s0	0000	0	s0	0000
s0	0000	1	s1	0001
s1	0001	0	s1	0001
s1	0001	1	s2	0010
s2	0010	0	s2	0010
s2	0010	1	s3	0011
s3	0011	0	s3	0011
s3	0011	1	s4	0100
s4	0100	0	s4	0100
s4	0100	1	s5	0101
s5	0101	0	s5	0101
s5	0101	1	s6	0110
s6	0110	0	s6	0110
s6	0110	1	s7	0111
s7	0111	0	s7	0111
s7	0111	1	s8	1000
s8	1000	0	s8	1000
s8	1000	1	s0	0000

Figure 2.3: State Table for FSM component.

State	Student ID # (Binary)	Current State (Binary)
s0	0101	0000
s1	0000	0001
s2	0001	0010
s3	0011	0011
s4	0001	0100
s5	0100	0101
s6	0110	0110
s7	0117	0111
s8	1001	1000

Figure 2.4: Output Table for FSM component.


```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY decoder3to8 IS
5  PORT (w : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
6        En : IN  STD_LOGIC;
7        y : OUT STD_LOGIC_VECTOR(0 TO 7));
8  END decoder3to8;
9
10 ARCHITECTURE Behaviour OF decoder3to8 IS
11     SIGNAL Enw : STD_LOGIC_VECTOR(3 DOWNTO 0);
12 BEGIN
13     Enw <= En & w;
14
15     WITH Enw SELECT
16 y <= "00000001" WHEN "1000",  -- s0 = 5
17     "00000010" WHEN "1001",  -- s1 = 0
18     "00000100" WHEN "1010",  -- s2 = 1
19     "00001000" WHEN "1011",  -- s3 = 3
20     "00010000" WHEN "1100",  -- s4 = 1
21     "00100000" WHEN "1101",  -- s5 = 4
22     "01000000" WHEN "1110",  -- s6 = 6
23     "10000000" WHEN "1111",  -- s7 = 7
24     "00000000" WHEN OTHERS;
25
26 END Behaviour;
27

```

Figure 3.1: VHDL Code for 3-to-8 Decoder.

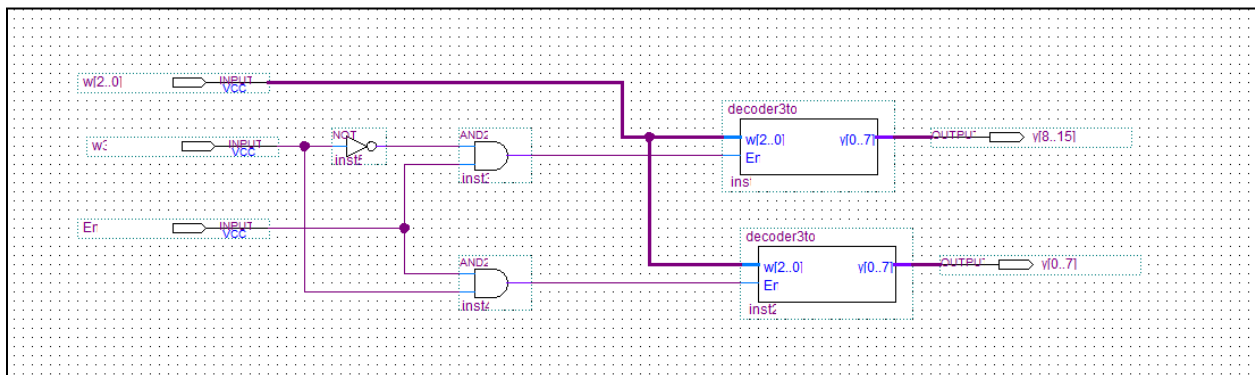


Figure 3.2: Block Diagram of 4-16 Decoder using two 3-to-8 Decoders.

En w0 w1 w2 w3	4-to-16 Decoder Output
10000	0000000000000001
10001	0000000000000010
10010	0000000000000100
10011	0000000000001000
10100	0000000000010000
10101	0000000000100000
10110	0000000001000000
10111	0000000010000000
11000	0000000100000000
11001	0000001000000000
11010	0000010000000000
11011	0000100000000000
11100	0001000000000000
11101	0010000000000000
11110	0100000000000000
11111	1000000000000000
00000	0000000000000000

Figure 3.3: Truth Table for 4-to-16 Decoder.

ALU_1 for Problem Set 1 of the Lab 6 procedure

The Arithmetic Logic Unit (ALU) core is the heart of every GPU unit in digital circuits. The ALU core is where all arithmetic and logic operations are performed as required to fulfill the purpose of the design. In the case of this lab, the ALU core took two 8-bit inputs (A and B), and a 16-bit microcode input from the 4-16 decoder described earlier. The purpose of the microcode

is this input is the *operation-selector signal*. This means that the type of Boolean operation that will be performed is directly determined by the microcode being sent into the ALU core. Despite the microcode received is 16 bits, only nine operations are to be completed to match the nine digits of the student ID number. These operations, along with their respective microcode, are located below in **Table 2**.

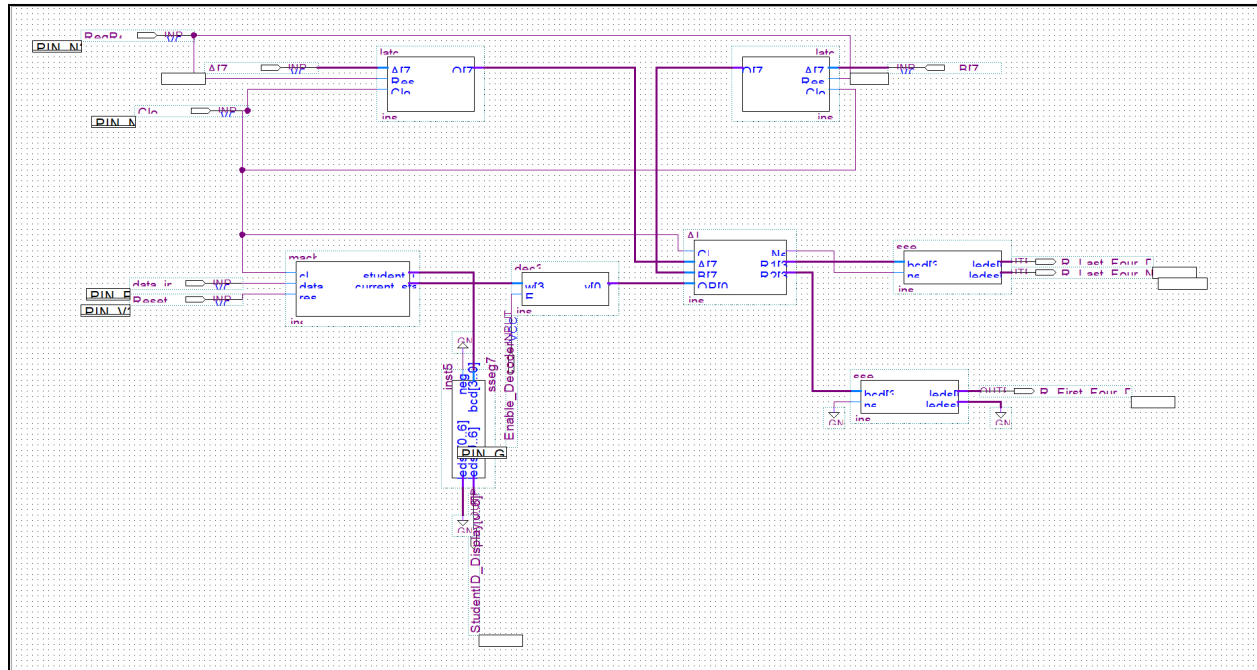


Figure 4: Block Diagram of ALU_1.

The purpose of all inputs and outputs of ALU_1 are as follows:

INPUTS

Clock → Clock signal, the ALU operations happen on each clock event. It synchronizes the ALU with the rest of the digital system.

A[7..0] → The first 8-bit data input to the ALU, it comes from Latch1 and it is used in all the arithmetic and logic operations.

B[7..0] → The second 8-bit data input to the ALU, it comes from Latch2 and it is used together with A in all the arithmetic and logic operations.

OP[0..15] → An 16-bit operation-selector from the decoder (microcode minterms), each bit corresponds to one ALU operation.

OUTPUTS

Neg → Indicates whether the result is negative (Neg = '0' means positive, Neg = '1' means negative display on the 7 segment).

R1[3..0] → Lower 4 bits of the 8-bit ALU result since the system can only display 4 bits at a time. It is sent to the 7 segment display on the FPGA.

R2[3..0] → Upper 4 bits of the 8-bit ALU result also sent to the 7 segment display on the FPGA.

R1 and R2 represent the full ALU result.

Table 1: ALU 1 nine microcodes generated by decoder, and its respective student ID digit.

Student ID #	Current State	Microcode (4-to-16 Decoder Output)
5	$s_0 = 0000$	0000000000000001
0	$s_1 = 0001$	0000000000000010
1	$s_2 = 0010$	0000000000000100
3	$s_3 = 0011$	0000000000001000
1	$s_4 = 0100$	0000000000010000
4	$s_5 = 0101$	0000000000100000
6	$s_6 = 0110$	0000000001000000
7	$s_7 = 0111$	0000000010000000
9	$s_8 = 1000$	0000000100000000

Table 2: Problem 1 ALU Microcode and Operations

Function #	Microcode	Boolean Operation/Function
1	0000000000000001	$sum(A, B)$
2	0000000000000010	$diff(A, B)$
3	0000000000000100	$\overline{A}$
4	0000000000001000	$\overline{A \cdot B}$
5	0000000000010000	$\overline{A + B}$
6	0000000000100000	$A \cdot B$
7	0000000001000000	$A \oplus B$
8	0000000010000000	$A + B$
9	0000000100000000	$\overline{A \oplus B}$

ALU_2 for Problem Set 2 of the Lab6 procedure

For ALU_2, the nine functions implemented in ALU_1 above were kept the same except for one. The one that had been changed was function 5. Instead of computing the NAND of inputs A and B, it was changed to instead compute the decrementation of B by 5. This was in the problem set table c) function 5, which can be found in the Lab 6 manual [Adapted from 1]. The purpose of this change was to directly modify the ALU core and its functionalities with an operation that is not a direct Boolean function like ALU_1. The decrementation of B by 5 requires knowledge of taking the 2's complement of bits and binary operations. The operation and its output is highlighted in yellow below in **Table 4** and **Figure 6**.

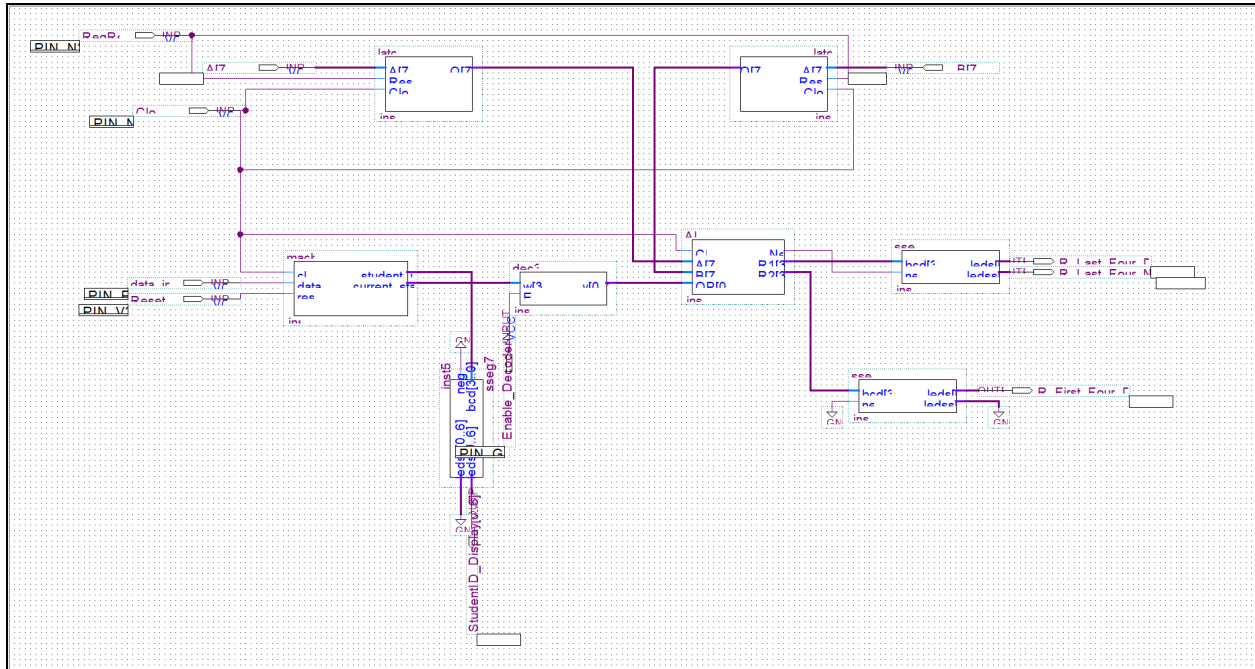


Figure 5: Block Diagram of ALU_2.

The purpose of all inputs and outputs of ALU_2 are as follows:

INPUTS

Clock → Clock signal, the ALU operations happen on each clock event. It synchronizes the ALU with the rest of the digital system.

A[7..0] → The first 8-bit data input to the ALU, it comes from Latch1 and it is used in all the arithmetic and logic operations.

B[7..0] → The second 8-bit data input to the ALU, it comes from Latch2 and it is used together with A in all the arithmetic and logic operations.

OP[0..15] → An 16-bit operation-selector from the decoder (microcode minterms), each bit corresponds to one ALU operation.

OUTPUTS

Neg → Indicates whether the result is negative (Neg = '0' means positive, Neg = '1' means negative display on the 7 segment).

R1[3..0] → Lower 4 bits of the 8-bit ALU result since the system can only display 4 bits at a time. It is sent to the 7 segment display on the FPGA.

R2[3..0] → Upper 4 bits of the 8-bit ALU result also sent to the 7 segment display on the FPGA. R1 and R2 represent the full ALU result.

Table 3: ALU 2 nine microcodes generated by decoder, and its respective student ID digit.

Student ID #	Current State	Microcode (4-to-16 Decoder Output)
5	$s_0 = 0000$	0000000000000001
0	$s_1 = 0001$	0000000000000010
1	$s_2 = 0010$	0000000000000100
3	$s_3 = 0011$	0000000000001000
1	$s_4 = 0100$	0000000000010000
4	$s_5 = 0101$	0000000000100000
6	$s_6 = 0110$	0000000001000000
7	$s_7 = 0111$	0000000010000000
9	$s_8 = 1000$	0000000100000000

Table 4: Problem 2 ALU Microcode and Operations

Function #	Microcode	Boolean Operation/Function
1	0000000000000001	$sum(A, B)$
2	0000000000000010	$diff(A, B)$
3	0000000000000100	$\overline{A}$
4	0000000000001000	$\overline{A \cdot B}$
5	0000000000010000	$diff(B, 5)$
6	0000000000100000	$A \cdot B$
7	0000000001000000	$A \oplus B$
8	0000000010000000	$A + B$
9	0000000100000000	$\overline{A \oplus B}$

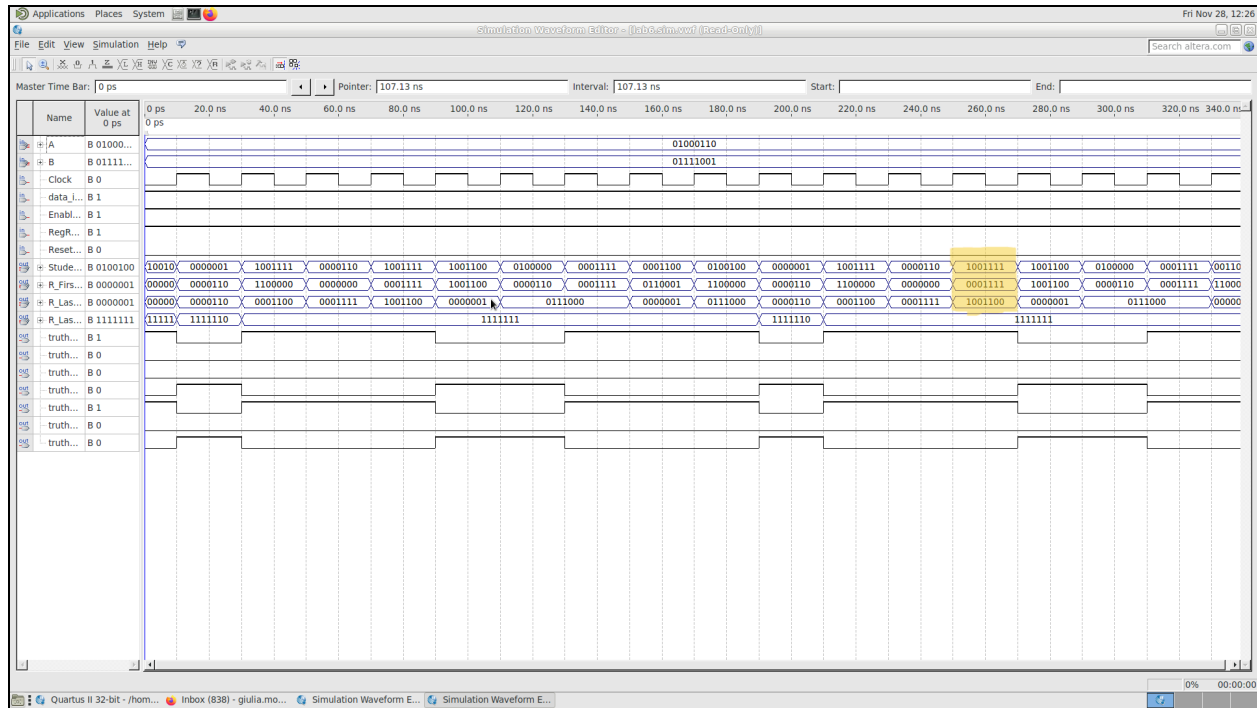


Figure 6: Problem 2 Waveform ALU_2.

ALU_3 for Problem Set 3 of the Lab6 procedure

For ALU_3, the student ID output from the FSM is now directly being utilized by the ALU core instead of simply being displayed as a sseg value on the FPGA. Another input was added to ALU_1, so that it can process four inputs instead of three. The 7 segment display file was also modified to display either “y” for yes or “n” for no as outputs instead of displaying HEX values produced by operations. Instead of Boolean operations with a numeric solution, the questions implemented in ALU_3 are simple yes or no questions regarding the student ID number, and possibly inputs A and B. Specifically, the question implemented below is Problem 3 question b) from the Lab 6 manual [Adapted from 1] which states:

“b) For each microcode instruction, display 'y' if the FSM output (student_id) is even and 'n' otherwise.”

The inputs and outputs upon implementation of this question into the ALU core are illustrated below in **Table 6** and **Figure 8**.

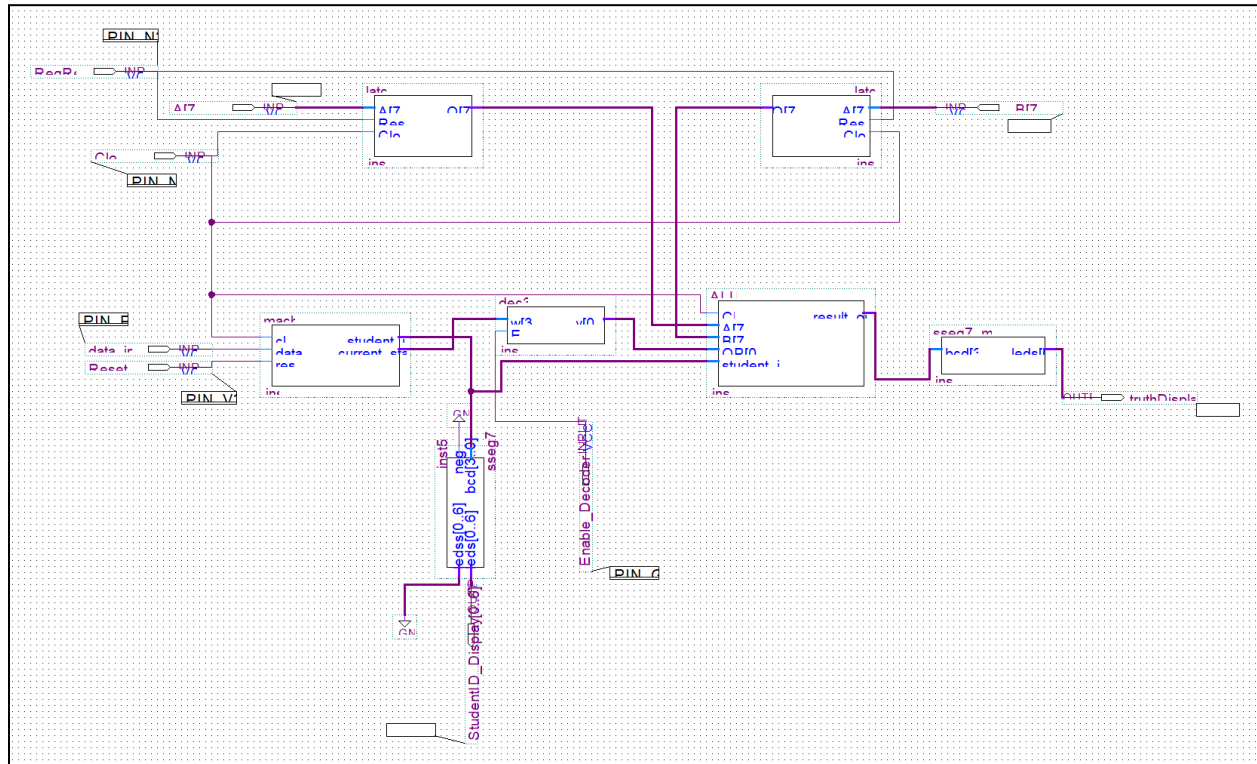


Figure 7: Block Diagram of ALU_3.

The purpose of all inputs and outputs of ALU_2 are as follows:

INPUTS

Clock → Clock signal, the ALU operations happen on each clock event. It synchronizes the ALU with the rest of the digital system.

A[7..0] → The first 8-bit data input to the ALU, it comes from Latch1 and it is used in all the arithmetic and logic operations.

B[7..0] → The second 8-bit data input to the ALU, it comes from Latch2 and it is used together with A in all the arithmetic and logic operations.

OP[0..15] → An 16-bit operation-selector from the decoder (microcode minterms), each bit corresponds to one ALU operation.

student_id[3..0] → 4-bit input coming from the FSM, represents one digit of the student ID at a time.

OUTPUTS

result_out[3..0] → 4-bit output of this component determining whether the FPGA will display “y” or “n” on the 7 segment display.

Table 5: ALU 3 nine microcodes generated by decoder, and its respective student ID digit.

Student ID #	Current State	Microcode (4-to-16 Decoder Output)
5	$s_0 = 0000$	0000000000000001
0	$s_1 = 0001$	0000000000000010
1	$s_2 = 0010$	0000000000000100
3	$s_3 = 0011$	0000000000001000
1	$s_4 = 0100$	0000000000010000
4	$s_5 = 0101$	0000000000100000
6	$s_6 = 0110$	0000000001000000
7	$s_7 = 0111$	0000000010000000
9	$s_8 = 1000$	0000000100000000

Table 6: Problem 3 ALU Microcode and Expected Outputs

Student ID #	Microcode	Y if Even (sseg 1000100) N if Odd (sseg 0001001)
5	0000000000000001	N
0	0000000000000010	Y
1	0000000000000100	N
3	0000000000001000	N
1	0000000000010000	N
4	0000000000100000	Y
6	0000000001000000	Y
7	0000000010000000	N
9	0000000100000000	N

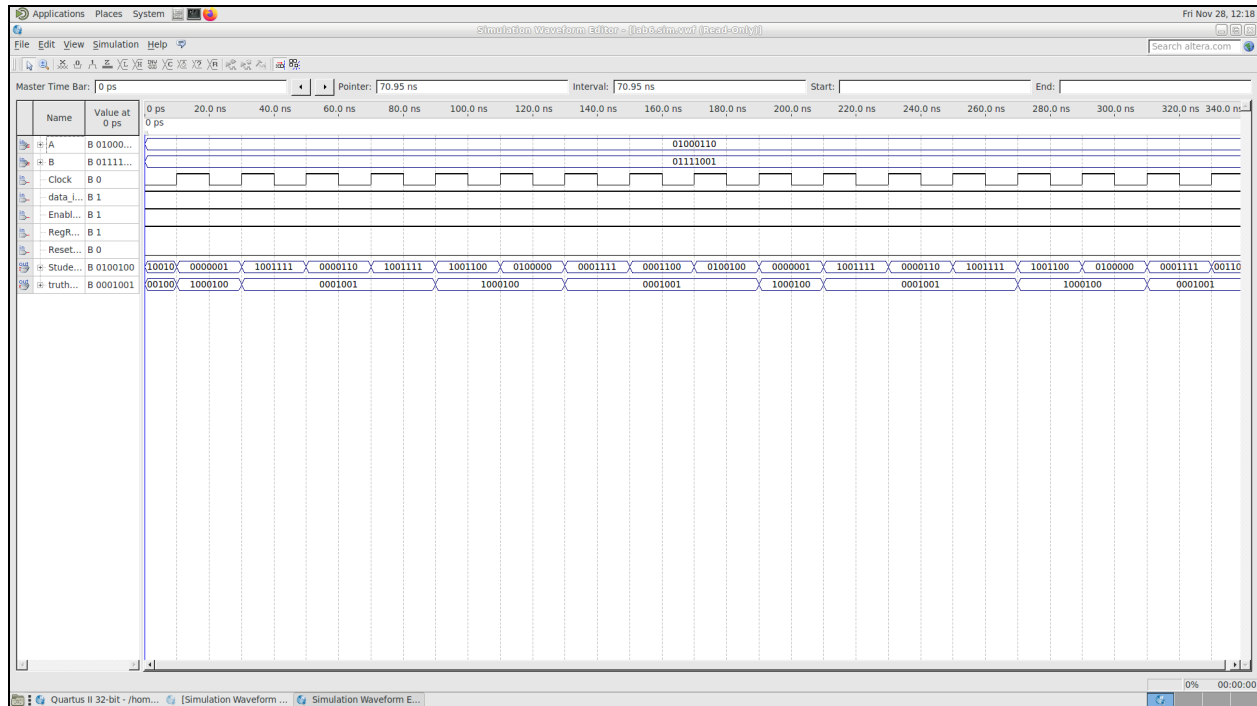


Figure 8: Problem 3 Waveform ALU Modified

Conclusion

To conclude, this project successfully explored, analyzed, designed, and implemented the Arithmetic and Logic Unit (ALU) along with its four distinct components. It was an excellent introduction to the concept of a GPU unit and its diverse applications in digital electronics, and why arithmetic operations being coordinated within a larger system is so important. Through this work, practical applications and experience of digital logic design, state machines, decoding, and data-path control were achieved, which is vital foundational knowledge for more advanced applications in the future, both in school and in the workplace. Overall, this lab highlighted the importance of ALU's and their essential role in complex processing and decision-making.

References

Department of Electrical and Computer Engineering. *Lab 6 - Design of a Simple*

General-Purpose Processor.

https://www.ee.torontomu.ca/~courses/coe328/Lab6_Manual.pdf